

Exercício 01: Sistema de E-commerce e Fretes

Contexto: Você está criando o carrinho de compras de uma loja virtual. O sistema precisa calcular fretes e permitir avaliações. No entanto, as avaliações não podem ser fora do padrão (1 a 5) e os preços não podem ser negativos.

Instruções:

1. **As Exceções:** * Crie uma classe `NotaInvalidaException` que herde de `Exception`.
 - Crie um construtor que receba uma mensagem e repasse para a superclasse.
2. **A Classe Base:** Crie uma classe abstrata `Produto`.
 - Atributos privados: `nome (String)` e `preco (double)`.
 - No método `setPreco(double preco)`, se o valor for menor que zero, lance uma `IllegalArgumentException` com a mensagem "O preço não pode ser negativo".
 - Método abstrato: `double calcularFrete()`.
3. **A Interface:** Crie a interface `Avaliavel`.
 - Contém o método `void adicionarAvaliacao(int nota) throws NotaInvalidaException`.
4. **As Classes Filhas:**
 - Crie `ProdutoFisico` (frete é **10%** do valor) e `ProdutoDigital` (frete **0.0**).
 - Ambas herdam de `Produto` e implementam `Avaliavel`. No método `adicionarAvaliacao`, se a nota for menor que 1 ou maior que 5, lance a `NotaInvalidaException`. Caso contrário, imprima a nota.
5. **Teste o Código:**
 - Na classe `Main`, instancie um carrinho (`List<Produto>`).
 - Peça ao usuário para adicionar produtos e digitar a avaliação. Envolve a chamada de `adicionarAvaliacao` e a definição de preço em um bloco `try-catch`, capturando as exceções e exibindo as mensagens de erro sem parar a execução do programa.
 - Ao final, itere sobre a lista e imprima o total da compra (produtos + frete).

Exercício 02: Gestão de Frota Logística (Exceções de Regra de Negócio)

Contexto: Ao gerenciar veículos refrigerados, a temperatura da carga não pode ultrapassar limites críticos que estraguem os produtos perecíveis.

Instruções:

1. **A Exceção:**
 - Crie a classe `TemperaturaCriticaException` (herda de `Exception`).
2. **A Classe Base:** Crie a classe abstrata `Veiculo`.
 - Atributos privados: `placa` e `capacidadeCarga`.
 - Métodos: `protected void registrarManutencao()` e o abstrato `void iniciarViagem()`.
3. **As Interfaces:**
 - `Rastreador`: método `String obterLocalizacao()`.
 - `Refrigerado`: método `void ajustarTemperatura(double graus)` throws `TemperaturaCriticaException`.
4. **As Classes Filhas:**
 - `CaminhaoPadrao` (herda `Veiculo`, implementa `Rastreador`).
 - `VanRefrigerada` (herda `Veiculo`, implementa `Rastreador` e `Refrigerado`).
 - Na `VanRefrigerada`, ao implementar `ajustarTemperatura`, se os graus forem maiores que `10.0` ou menores que `-20.0`, lance a `TemperaturaCriticaException` informando que a carga pode ser comprometida.
5. **Teste o Código:**
 - Instancie os veículos no `Main`.
 - Crie um bloco `try-catch` para tentar ajustar a temperatura da Van para `15.0` (o que deve gerar o erro) e outro para ajustar para `2.0` (o que deve funcionar).

Exercício 03: Sistema Acadêmico (Validação de Dados)

Contexto: O sistema universitário precisa garantir que notas inválidas não sejam lançadas no histórico dos alunos e que artigos sem título não sejam publicados.

Instruções:

1. **A Exceção:**
 - Crie a classe `DadosAcademicosInvalidosException` (herda de `RuntimeException`).
2. **A Classe Base:** Crie a classe abstrata `MembroAcademico`.
 - Atributos privados: `nome` e `matricula`.
 - Método abstrato: `void mostrarFuncao()`.
3. **As Interfaces:**
 - `Pesquisador`: `void publicarArtigo(String titulo)`.
 - `Avaliador`: `void lancarNotas(double nota)`.
4. **As Classes Filhas:**
 - `Professor`: Implementa ambas as interfaces. Em `lancarNotas`, se a nota for menor que 0 ou maior que 10, lance a `DadosAcademicosInvalidosException`.
 - `AlunoPosGraduacao`: Implementa `Pesquisador`. Em `publicarArtigo` (para ambas as classes), se o título for vazio ou nulo, lance também a exceção.
5. **Teste o Código:**
 - No Main, crie uma `List<MembroAcademico>`. Adicione professores e alunos.
 - Itere sobre a lista usando `instanceof`. Envolve as chamadas de publicar artigo (passe uma string vazia para testar) e lançar notas (passe um valor `11.0`) em blocos `try-catch` para garantir que o programa trate o erro e continue a execução.

Exercício 04: Jogo de RPG - Sistema de Batalha Seguro

Contexto: Em um sistema de batalha, personagens não podem ser atacados ou curados se já estiverem derrotados (Vida = 0).

Instruções:

1. As Exceções:

- Crie `AlvoInvalidoException` (herda de `Exception`).

2. A Classe Base: Personagem.

- Atributos privados: `nome` e `pontosVida`. O Setter de vida não deve permitir valores abaixo de zero.
- Método abstrato: `void atacar(Personagem alvo) throws AlvoInvalidoException`.

3. As Interfaces:

- `Magico`: `void lancarFeitico(Personagem alvo) throws AlvoInvalidoException`.
- `Curandeiro`: `void curarAliado(Personagem aliado) throws AlvoInvalidoException`.

4. As Classes Filhas:

- `Guerreiro`, `Paladino` e `Mago`.
- Em todas as implementações de ataques e curas, faça uma verificação: se o alvo (ou aliado) tiver 0 pontos de vida, lance a `AlvoInvalidoException` com a mensagem "[Nome do alvo] já está fora de combate!".

5. Teste o Código:

- No Main, instancie os personagens.
- Simule ataques repetidos em um bloco de repetição até a vida de um personagem chegar a zero.
- Dentro de um try-catch, tente atacar o personagem que já está com a vida zerada para forçar o disparo e a captura da exceção.

Exercício 05: Plataforma de Streaming de Mídia (Simulação de Rede)

Contexto: Downloads podem falhar devido a problemas de conexão ou falta de permissão na conta do usuário.

Instruções:

1. **A Exceção:**
 - Crie `FalhaNoDownloadException` (herda de `Exception`).
2. **A Classe Base:** `Midia`.
 - Atributos privados: `titulo` e `duracaoSegundos`.
 - Método abstrato: `void reproduzir()`.
3. **As Interfaces:**
 - `Baixavel`: `void realizarDownload(boolean conexaoEstavel) throws FalhaNoDownloadException`.
 - `Compartilhavel`: `String gerarLink()`.
4. **As Classes Filhas:**
 - `Musica` (implementa ambas), `Filme` (implementa `Baixavel`), `Podcast` (implementa `Compartilhavel`).
 - Nas classes que implementam `Baixavel`, se o parâmetro `conexaoEstavel` for `false`, lance a `FalhaNoDownloadException` ("Erro de rede: Não foi possível baixar [titulo]").
5. **Teste o Código:**
 - No `Main`, crie a lista `playlist` e adicione as mídias.
 - Itere sobre a lista. Quando a mídia for `Baixavel`, chame `realizarDownload(false)` deliberadamente para forçar o erro de conexão.
 - Envolve a chamada em um `try-catch` para que, se o download falhar, o programa avise o usuário na tela, mas continue reproduzindo as próximas mídias da `playlist`.